

KernelMind: The Hybrid Meta-Scheduler

Assignment 3 Report - CPU Scheduling Meta-Agent

This report accompanies the notebook and code archive. It describes the implementation, metrics, reward design, Q-learning agent, and the observed performance of the trained meta-scheduler. The experiment used 20,000 training episodes and a 500-episode held-out evaluation set generated from the mixed workload distribution.

Implementation Overview

Part 1. The code defines a Process dataclass that tracks pid, arrival time, burst time, remaining time, start time, finish time, and executed CPU ticks. The workload generator supports uniform, convoy, I/O storm, and mixed episodes. The baseline simulators are independent loops for FCFS, non-preemptive SJF, RR with quantum 4, and a Random Agent that selects one baseline heuristic uniformly at random per episode.

Part 2. The evaluation utility computes mean wait time, P90 wait time, Jain fairness index over wait times, and makespan. Jain fairness is computed as $(\sum x_i)^2 / (n * \sum x_i^2)$, with a value of 1.0 used when all waits are zero.

Part 3. The MDP state is a tuple of discretized queue length, average remaining burst bucket, and maximum current-wait bucket. The action space is FCFS, SJF, and RR. Each step executes one CPU tick, so the meta-agent can switch heuristics dynamically.

Part 4. The agent is a tabular Q-learning agent using the Bellman update, epsilon-greedy exploration, random tie-breaking, exponential epsilon decay, and a hard cutoff to pure exploitation late in training.

Part 5. The experiment trains the agent, evaluates baselines and the trained policy on identical held-out workloads, saves the Q-table, exports metrics to CSV, and creates a convergence plot with baseline mean-wait reference lines.

Reward Function

The per-step reward is:

```
r_t = -(0.035 * total_queue_delay + 0.80 * (max_wait / 10)^2 + context_switch_cost *
switched)
```

At terminal completion, the environment adds:

```
-(0.30 * mean_wait + 0.08 * p90_wait + 12.0 * (1 - Jain_fairness))
```

The total-delay term encourages throughput, while the quadratic max-wait term makes starvation increasingly expensive. The terminal P90 and Jain terms penalize poor tail latency and unequal waiting-time distribution.

Comparative Evaluation Results

Policy	Mean Wait	P90 Wait	Jain Fairness	Makespan
RL Meta-Scheduler	25.5176	50.7954	0.6384	100.188
RR(q=4)	37.6790	54.6386	0.8127	100.188
SJF	40.2556	66.4632	0.6968	100.188
Random Agent	41.8442	64.1144	0.7656	100.188
FCFS	49.8962	73.3022	0.7956	100.188

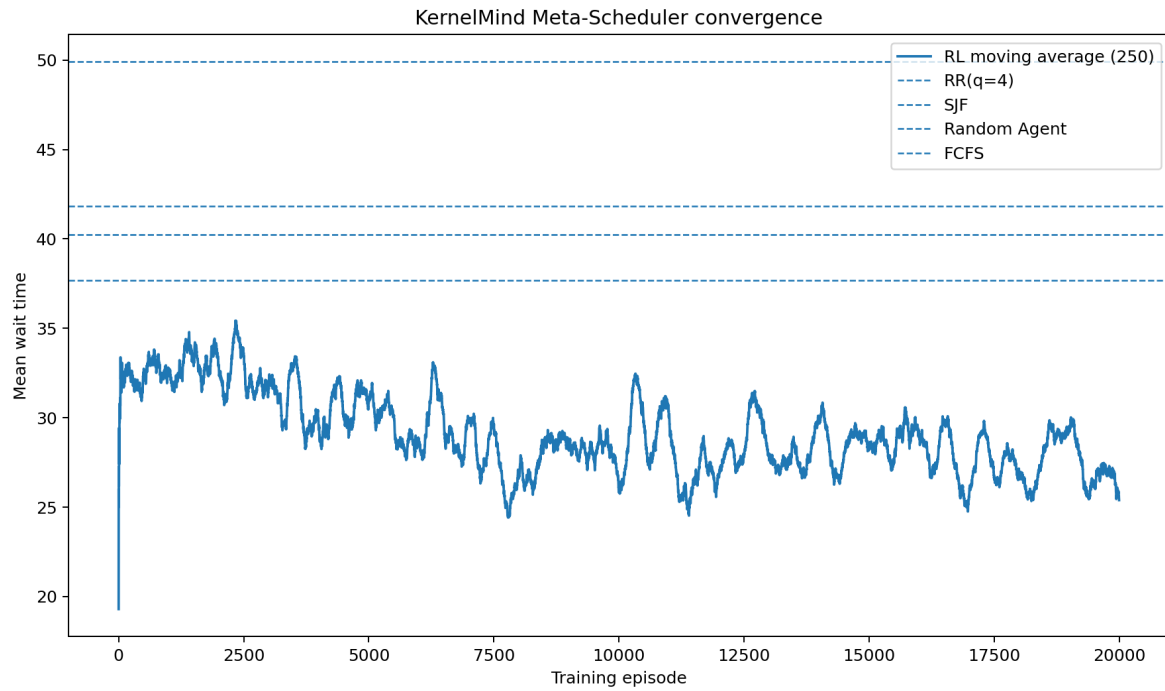
The RL Meta-Scheduler produced the best mean wait time and P90 wait time. Its Jain fairness was lower than RR because the reward emphasized throughput and tail latency rather than perfectly equal wait distribution.

Greedy Evaluation Action Distribution

Action	Count	Fraction
FCFS	8403	0.1691
SJF	19740	0.3972
RR	21560	0.4338

The agent used all three actions: RR 43.38%, SJF 39.72%, and FCFS 16.91%. This demonstrates that it learned a hybrid policy rather than collapsing into a single static algorithm.

Convergence Plot



The moving average of training mean wait drops below the static baseline reference lines, indicating that the learned policy becomes competitive with and then stronger than the baselines on the mixed workload distribution.

Design Question 1: The Preemption Paradigm

Preemption is mathematically necessary for the absolute lowest mean wait time because the scheduler must be able to revise its choice when a shorter remaining job appears. The exchange argument is that if job A has a long remaining time and job B has a shorter remaining time, making B wait behind A increases B's completion time more than it helps A. Running B first reduces the sum of completion times and therefore reduces average waiting time. Shortest Remaining Time First applies that rule continuously.

The flaw in non-preemptive SJF is that it only chooses when the CPU becomes free. If a massive database backup starts one tick before a dozen tiny web requests arrive, the web requests must wait for the backup to finish. This creates the convoy effect. SRTF would interrupt the backup, serve the tiny requests quickly, and then resume the backup.

Design Question 2: Reward Shaping

A simple linear penalty can let the agent justify starving one large job if many smaller jobs benefit. The reward therefore combines linear aggregate delay with a quadratic maximum-wait term. Because the maximum-wait term is squared, the cost of leaving one process behind grows superlinearly. Eventually that one starving process dominates the reward loss, so the agent cannot keep improving its return by ignoring it indefinitely. The terminal P90 and Jain penalties further discourage tail-latency abuse and unfair final waiting-time distributions.

Design Question 3: Output Analysis

The final table shows that the trained agent formed a hybrid policy. It achieved a mean wait time of 25.5176, compared with 37.6790 for RR, 40.2556 for SJF, 41.8442 for the Random Agent, and 49.8962 for FCFS. It also had the best P90 wait time at 50.7954. This means the agent improved both throughput and tail latency on the evaluation distribution.

The action distribution explains how this happened. The policy selected RR most often, then SJF, then FCFS. SJF-like choices reduce average wait by favoring short jobs, while RR gives long-waiting jobs service and prevents the worst starvation cases. FCFS appears mainly in states where the queue is small or arrival order is already a reasonable choice. Jain fairness remained below pure RR, so the agent did not optimize equality alone; it learned the intended throughput-tail-latency compromise.

Submitted Files

KernelMind_Assignment3.ipynb, kernelmind_assignment3.py, report.pdf, convergence_plot.png, comparative_metrics.csv, training_history.csv, action_distribution.csv, all_episode_metrics.csv, q_table.npy, README.md, and the ZIP archive.